

Taller 1 IA

Integrantes:

Baruc jeronimo Triana Bastidas 202416093

Juan Avendaño 202411658

Sara Hernández Orjuela 202412681

Punto 1

Diagnóstico inicial mediante DFS

Primera implementación:

```
def depthFirstSearch(problem: SearchProblem):
    """
    Search the deepest nodes in the search tree first.

    Your search algorithm needs to return a list of actions that reaches the
    goal. Make sure to implement a graph search algorithm.

    To get started, you might want to try some of these simple commands to
    understand the search problem that is being passed in:

    print("Start:", problem.getStartState())
    print("Is the start a goal?", problem.isGoalState(problem.getStartState()))
    print("Start's successors:",
          problem.getSuccessors(problem.getStartState()))
    """
    stack=utils.Stack()

    print("Start:", problem.getStartState())

    stack.push(problem.getStartState(),[])

    visited=set()

    while not stack.isEmpty():
        state,actions=stack.pop()

        if state in visited:
            continue

        visited.add(state)
```

```

    if problem.isGoalState(state):
        return actions

    for successor, action, stepCost in problem.getSuccessors(state):

        if successor not in visited:
            stack.push((successor, actions + [action]))

return []

```

Cuando ejecuté el algoritmo me salió un error que chatgpt me ayudó a corregir, el primero fue en la línea `stack.push(problem.getStartState(),[])`. El error era el siguiente: `line 35, in depthFirstSearch stack.push(problem.getStartState(),[]) TypeError: push() takes 2 positional arguments but 3 were given`. La explicación de la IA fue recordarme el correcto funcionamiento de la función `push` de la clase `stack`. Puesto que en `def push(self, item):` el `self` lo proporciona automáticamente Python, mientras que `item` es el único argumento que yo puedo proporcionar, en este caso una tupla.

Incorrecto

Python

```
stack.push(problem.getStartState(), [])
```



Ejecutar

Aquí estás pasando dos argumentos.

Correcto

Python

```
stack.push((problem.getStartState(), []))
```



Ejecutar

Versión de chatgpt:

```

def depthFirstSearch(problem: SearchProblem):
    """
    Search the deepest nodes in the search tree first.

    Your search algorithm needs to return a list of actions that reaches the
    goal. Make sure to implement a graph search algorithm.

    To get started, you might want to try some of these simple commands to
    understand the search problem that is being passed in:

    print("Start:", problem.getStartState())
    print("Is the start a goal?", problem.isGoalState(problem.getStartState()))
    print("Start's successors:",
    problem.getSuccessors(problem.getStartState()))

```

```

"""
stack=utils.Stack()

print("Start:", problem.getStartState())

stack.push((problem.getStartState(),[]))

visited=set()

while not stack.isEmpty():
    state,actions=stack.pop()

    if state in visited:
        continue

    visited.add(state)

    if problem.isGoalState(state):
        return actions

    for successor, action, stepCost in problem.getSuccessors(state):

        if successor not in visited:
            stack.push((successor, actions + [action]))

return []

```

Punto 2

Ruta más corta mediante BFS

Es el mismo algoritmo de antes, sólo que en vez de usar una pila se usa una cola.

```

def breadthFirstSearch(problem: SearchProblem):
    """
    Search the shallowest nodes in the search tree first.
    """
    queue=utils.Queue()

    print("Start:", problem.getStartState())

    queue.push((problem.getStartState(),[]))

    visited=set()

    while not queue.isEmpty():

```

```
state,actions=queue.pop()

if state in visited:
    continue

visited.add(state)

if problem.isGoalState(state):
    return actions

for successor, action, stepCost in problem.getSuccessors(state):

    if successor not in visited:
        queue.push((successor, actions + [action]))

return []
```

Punto 3

- a) Empecé implementando la función del UCS, de manera muy similar a los anteriores recorridos, ya que es una implementación semejante, solo que con otra estructura de datos (Una cola de prioridad), sin embargo al terminar de implementarla y al tratar de probarla, note un error al hacer un push a la cola de prioridad, ya que no le estaba pasando todos los parametros necesarios. (Imagen de implementación previa a la IA debajo)

```

90
91 def uniformCostSearch(problem: SearchProblem):
92     """
93     Search the node of least total cost first.
94     """
95
96     priority_queue = utils.PriorityQueue()
97
98     print("Start:", problem.getStartState())
99     priority_queue.push((problem.getStartState(), []), 0)
100
101     visited = set()
102
103     while not priority_queue.isEmpty():
104
105         state, actions, cost = priority_queue.pop()
106
107         if state in visited:
108             continue
109
110         visited.add(state)
111
112         if problem.isGoalState(state):
113             return actions
114

```

Sin embargo, yo no me di cuenta de este error por mi cuenta, para esto le pregunte a la IA mediante el siguiente pront (habiendole pasado el contexto del taller anteriormente). “Necesito entender porque no esta funcionando mi implementación del punto 3, el UCS, ya que como tengo entendido es simplemente hacer una implementación similar a los anteriores recorridos. Además no entiendo del todo el error, ya que me dice que no hay suficientes valores para desenpaquetar, pero no entiendo a que se refiere. Este es el error exacto: File "/Users/baructriana/Documents/Sexto_Semestre/IA/Taller-1-Inteligencia-Artificial/EstacionEspacial/algorithms/search.py", line 105, in uniformCostSearch

```
state, actions, cost = priority_queue.pop()
```

ValueError: not enough values to unpack (expected 3, got 2)

Y chat me explico el error de manera detallada (primero pidiendome que le pasara la estructura completa ya implementada de la cola de prioridad) y me dijo que añadiera los parametros necesarios en el push y que cambiara mi logica para que despues se hiciera el push para cada sucesor.

Imagen de implementacion final UCS

```

def uniformCostSearch(problem: SearchProblem):
    """
    Search the node of least total cost first.
    """
    priority_queue = utils.PriorityQueue()

    print("Start:", problem.getStartState())
    priority_queue.push((problem.getStartState(), [], 0), 0)
    visited = set()
    while not priority_queue.isEmpty():
        state, actions, cost = priority_queue.pop()
        if state in visited:
            continue
        visited.add(state)

        if problem.isGoalState(state):
            return actions
        for successor, actual_action, stepcost in problem.getSuccessors(state):
            if successor not in visited:
                new_cost = cost + stepcost
                priority_queue.push((successor, actions + [actual_action], new_cost), new_cost)

    return []

```

Punto 4

En cuanto al punto 4, en los numerales A y B no tuve problemas, ya que su implementación era muy similar a la usada anteriormente de los otros problemas, solo que añadiéndole un booleano (el de tener o no tener el modulo) donde algunas condiciones cambiaban, como en el de isGoalState, solo se tuvo que agregar una condición de revisar el booleano en el return. Y en el punto B, el de la función de _getStepCost solo se tuvo que implementar una logica donde preguntara si primero tenia ya el modulo, ya que si, si el costo de las acciones se multiplicaba por 2 y si no, el costo se mantenía igual.

```

def isGoalState(self, state):
    """
    Returns True if the robot reached C after picking up M.
    """
    position, hasModule = state

    return position == self.controlPosition and hasModule

def _getStepCost(self, nextPosition, hasModule):
    """
    Returns the movement cost for entering nextPosition.
    """

    costo_normal= self.startingMissionState.getTerrainCost(nextPosition[0], nextPosition[1])

    if hasModule:
        return costo_normal * 2
    else:
        return costo_normal

```

Sin embargo para el numeral C, el de la función de getSuccesors, si tuve un problema ya que mi implementación inicial, a pesar de seguir siendo un poco similar al getSuccesors normal, se complicaba al tener revisar si tenia el modulo, por lo que cometi un error conceptual al declarar el costo (stepcost) con nextPosition y nextHasModule. Sin embargo esto no me genero un error, pero al no estar tan seguro de la correctitud de esta función, le pregunte a la IA el siguiente pront: “Estaba haciendo el punto 4, y creo que implemente de manera correcta las funciones del numeral A y B, sin embargo no estoy muy seguro de la C. Necesito que revisas las 3 funciones con le contexto del proyecto y que me digas que deberia cambiar y explicandome el porque (Adjunte las 3 funciones)” y la IA respondio esto (adjunto imagen de codigo con el error):

```

def getSuccessors(self, state):
    Use the auxiliary function defined above to calculate the cost of each step.

    Mission rules:
    - The robot picks up the module when it enters M.
    - Movement uses normal terrain cost while the robot is not carrying M.
    - Once the robot is carrying M, movement costs twice the normal terrain cost
    - The movement used to enter M still has normal terrain cost.
    """

    successors = []
    self._expanded += 1

    position, hasModule = state

    for direction in [
        Directions.NORTH,
        Directions.SOUTH,
        Directions.EAST,
        Directions.WEST,
    ]:
        x, y = position

        dx, dy = Actions.directionToVector(direction)

        nextx, nexty = int(x + dx), int(y + dy)

        if not self.walls[nextx][nexty]:
            nextPosition = (nextx, nexty)

            if hasModule:
                nextHasModule = True
            elif nextPosition == self.modulePosition:
                nextHasModule = True
            else:
                nextHasModule = False
            stepcost = self._getStepCost(nextPosition, nextHasModule)

            nextState = (nextPosition, nextHasModule)

            successors.append((nextState, direction, stepcost))
    return successors

```


Respuesta de la IA

Revisando las tres funciones del punto 4 con las reglas de `ModuleRepairProblem`, las funciones de los numerales A y B están correctamente planteadas, pero encontré un error lógico en `getSuccessors`.

A. `isGoalState`

Tu implementación:

```
</> Python Ejecutar  
  
def isGoalState(self, state):  
    position, hasModule = state  
  
    return position == self.controlPosition and hasModule
```

está correcta.

C. `getSuccessors`

Aquí encontré el problema.

Tienes esta parte:

```
</> Python Ejecutar  
  
if hasModule:  
    nextHasModule = True  
elif nextPosition == self.modulePosition:  
    nextHasModule = True  
else:  
    nextHasModule = False  
  
stepCost = self._getStepCost(  
    nextPosition,  
    nextHasModule  
)
```

El cálculo de `nextHasModule` está bien. El problema está en utilizar:

```
</> Python Ejecutar  
  
nextHasModule
```

para calcular el costo del movimiento.

La instrucción debería usar:

```
</> Python Ejecutar  
  
hasModule
```

¿Por qué?

`hasModule` representa si el robot tenía el módulo **antes de realizar el movimiento**.

En cambio:

Python

```
nextHasModule
```



Ejecutar

representa si el robot tendrá el módulo **después de realizar el movimiento**.

Esto es importante cuando el robot entra justamente a la casilla **M**.

Por ejemplo, antes de entrar a **M**:

Python

```
hasModule = False
```



Ejecutar

Después de entrar:

Python

```
nextHasModule = True
```



Ejecutar

Según las reglas del taller, el movimiento utilizado para entrar a **M** todavía debe tener su costo normal. El costo doble comienza a aplicarse a partir del movimiento siguiente.

Por lo que a partir de eso, solo tuve que cambiar el `stepCost` y ya quedo funcional y cumple con lo que pedía el numeral C y el contexto del taller. (Adjunto imagen del código ya arreglado)

```

class ModuleRepairProblem(SearchProblem):
    def getSuccessors(self, state):
        Use the auxiliary function defined above to calculate the cost of each step.

        Mission rules:
        - The robot picks up the module when it enters M.
        - Movement uses normal terrain cost while the robot is not carrying M.
        - Once the robot is carrying M, movement costs twice the normal terrain cost.
        - The movement used to enter M still has normal terrain cost.
        """

        successors = []
        self._expanded += 1

        position, hasModule = state

        for direction in [
            Directions.NORTH,
            Directions.SOUTH,
            Directions.EAST,
            Directions.WEST,
        ]:
            x, y = position

            dx, dy = Actions.directionToVector(direction)

            nextx, nexty = int(x + dx), int(y + dy)

            if not self.walls[nextx][nexty]:
                nextPosition = (nextx, nexty)

                if hasModule:
                    nextHasModule = True
                elif nextPosition == self.modulePosition:
                    nextHasModule = True
                else:
                    nextHasModule = False
                stepcost = self._getStepCost(nextPosition, hasModule)

                nextState = (nextPosition, nextHasModule)

                successors.append((nextState, direction, stepcost))
        return successors

```

Punto 5

a)

Código Inicial A*:

```

def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):
    """
    Search the node that has the lowest combined cost and heuristic first.
    """
    pqueue = utils.PriorityQueue()
    start_state = problem.getStartState()
    start_h = heuristic(start_state, problem)

    pqueue.push((start_state, [], 0), start_h)
    best_g = {start_state: 0}

    while not pqueue.isEmpty():
        state, actions, cost = pqueue.pop()

        if cost > best_g.get(state, float("inf")):
            continue

        if problem.isGoalState(state):
            return actions

        for successor, action, stepCost in problem.getSuccessors(state):
            new_cost = cost + stepCost

            if successor not in best_g or new_cost < best_g[successor]:
                best_g[successor] = new_cost
                priority = new_cost + heuristic(successor, problem)
                pqueue.push((successor, actions + [action], new_cost), priority)

    return []

```

Prompt:

(Código A* adjunto)

Estoy resolviendo el proyecto "Estación Espacial Aurora": un robot se mueve en una cuadrícula y hay que implementar A*.

Necesito que:

1. Dime si el código es correcto (admisible y consistente).
2. Si no es correcto, dime exactamente qué falla y dame la corrección.
3. Si es correcto, pero existe una versión mejor, dame la mejor y la razón de por qué es mejor.

Respuesta:

La implementación de aStarSearch se conserva sin modificaciones porque cumple correctamente las tres condiciones que garantizan el comportamiento esperado del algoritmo. En primer lugar, la frontera se ordena por $f(n) = g(n) + h(n)$ mediante una cola de prioridad, mientras que la prueba de meta se realiza en el momento de la expansión y no en el de la generación; dado que el nodo extraído posee la f mínima de toda la frontera y las heurísticas empleadas son admisibles, ningún camino pendiente puede mejorar ese costo, por lo que la primera meta extraída es necesariamente la óptima. En segundo lugar, el diccionario best_g registra el menor costo real conocido por estado y la comparación de reapertura se efectúa

sobre $g(n)$ puro y no sobre $f(n)$; esta separación permite reabrir un estado ya visitado cuando más adelante se descubre un camino de menor costo, de modo que la optimalidad se mantiene con cualquier heurística admisible, sin exigir adicionalmente que sea consistente —requisito habitual de A* sobre grafos—. En tercer lugar, como la cola de prioridad del proyecto no admite reducir la prioridad de un elemento ya insertado, el algoritmo inserta una nueva entrada y descarta la obsoleta al extraerla mediante la verificación `if cost > best_g.get(state, inf):` continúe, lo que evita implementar una operación de decrease-key sobre el montículo sin comprometer la correctitud. cualquier reescritura constituiría un cambio puramente cosmético sin beneficio medible.

b)

Código Inicial Manhattan:

```
def manhattanHeuristic(state, problem):
    """Manhattan distance heuristic towards the next mandatory target."""
    position, hasKit, pendingSystems = state

    if not hasKit:
        target = problem.kitPosition
        return abs(position[0] - target[0]) + abs(position[1] - target[1])
    elif pendingSystems:
        return min(
            abs(position[0] - sys[0]) + abs(position[1] - sys[1])
            for sys in pendingSystems
        )
    else:
        target = problem.controlPosition
        return abs(position[0] - target[0]) + abs(position[1] - target[1])
```

Prompt:

(Código manhattan adjunto)

Toca implementar la heurística de la distancia de manhattan

Necesito que:

1. Dime si el código es correcto (admisible y consistente).
2. Si no es correcto, dime exactamente qué falla y dame la corrección.
3. Si es correcto, pero existe una versión mejor, dame la mejor y la razón de por qué es mejor.

Respuesta:

Propiedades de la heurística

- Admisible: Sí. En un mapa de cuadrícula donde los movimientos son ortogonales y cuestan 1 paso, la distancia Manhattan representa la ruta más corta teórica ignorando obstáculos. Como el robot debe visitar de forma obligatoria el kit, los sistemas y el

centro de control en ese orden estricto, la distancia hasta el siguiente objetivo obligatorio nunca sobreestimar  el costo real restante: $h(n) \leq h^*(n)$.

Consistente: S . Cumple la desigualdad triangular $h(n) \leq c(n, a, n') + h(n')$. Dentro de una misma fase, dar un paso cambia la distancia Manhattan en como m ximo 1 paso (igual al costo $c = 1$). En los cambios de fase (al llegar a la celda del kit o de un sistema), la heur stica en el nodo actual vale 0, por lo que $0 \leq 1 + h(n')$ se cumple trivialmente pues $h(n') \geq 0$.

C digo corregido y optimizado

```
def _manhattanDistance(a, b):  
    """Distancia Manhattan |dx| + |dy| entre dos celdas."""  
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
def _phaseDistance(state, problem, distance):  
    """  
    Aplica la regla de fase y devuelve la distancia al objetivo obligatorio.  
  
    state      : (position, hasKit, pendingSystems)  
    problem    : SystemRepairProblem (aporta kitPosition y controlPosition)  
    distance   : funcion de distancia a usar (_manhattanDistance o _euclideanDistance)  
    """  
  
    position, hasKit, pendingSystems = state  
  
    if not hasKit:  
        return distance(position, problem.kitPosition)  
  
    if pendingSystems:  
        return min(distance(position, system) for system in pendingSystems)  
  
    return distance(position, problem.controlPosition)
```

```
def manhattanHeuristic(state, problem):  
    """  
    Punto 5b. Distancia Manhattan hasta el objetivo obligatorio de la fase.  
  
    Admisible: el robot solo se mueve en 4 direcciones y cada paso cuesta 1,  
    asi que la distancia Manhattan nunca supera el numero real de pasos hasta  
    esa celda; y esa celda es de visita obligatoria antes de terminar la  
    mision. Por lo tanto  $h(n) \leq h^*(n)$ .  
  
    Consistente: dentro de una misma fase, un paso cambia la distancia  
    Manhattan en a lo sumo 1, luego  $h(n) \leq 1 + h(n')$ . En los pasos que cambian  
    de fase (entrar a K o reparar un T) el termino que desaparece vale  
    exactamente 1, de modo que se obtiene la igualdad y no una violacion.  
    """  
    return _phaseDistance(state, problem, _manhattanDistance)
```

 Por qu  es mejor esta versi n?

- Principio DRY (Don't Repeat Yourself): Elimina la duplicación de la fórmula $\text{abs}(x1 - x2) + \text{abs}(y1 - y2)$ que en el código original se repetía tres veces.
- Facilidad de Pruebas Unitarias: Aislar el cálculo de la distancia en la función auxiliar `_manhattanDistance` permite probar esa matemática de forma independiente sin necesidad de simular un objeto state o problem.
- Mantenibilidad: Si las posiciones cambian de formato en el futuro (por ejemplo, de tuplas 2D a objetos con atributos `.x` y `.y`), solo modificas la función auxiliar en lugar de cambiar múltiples líneas en la heurística principal.

c)

Código Inicial Euclidean:

```
def euclideanHeuristic(state, problem):
    """Euclidean distance heuristic towards the next mandatory target."""
    position, hasKit, pendingSystems = state

    if not hasKit:
        target = problem.kitPosition
        return (
            (position[0] - target[0]) ** 2 + (position[1] - target[1]) ** 2
        ) ** 0.5
    elif pendingSystems:
        return min(
            ((position[0] - sys[0]) ** 2 + (position[1] - sys[1]) ** 2) ** 0.5
            for sys in pendingSystems
        )
    else:
        target = problem.controlPosition
        return (
            (position[0] - target[0]) ** 2 + (position[1] - target[1]) ** 2
        ) ** 0.5
```

Prompt:

(Código euclidean adjunto)

Toca implementar la heurística de la distancia euclidiana

Necesito que:

1. Dime si el código es correcto (admisible y consistente).
2. Si no es correcto, dime exactamente qué falla y dame la corrección.
3. Si es correcto, pero existe una versión mejor, dame la mejor y la razón de por qué es mejor.

Respuesta:

Propiedades de la heurística

- Admisible: Sí. La distancia euclidiana es la distancia en línea recta entre dos puntos:

$$d = \sqrt{\Delta x^2 + \Delta y^2}.$$

En una cuadrícula donde los movimientos son ortogonales y cuestan 1 paso, la distancia euclidiana siempre es menor o igual a la distancia Manhattan y, por ende, siempre es menor o igual al costo real del camino $h^*(n)$. Jamás sobreestima el costo restante: $h(n) \leq h^*(n)$.
- Consistente: Sí. Satisface la desigualdad triangular $h(n) \leq c(n, a, n') + h(n')$.
 Un movimiento de una celda adyacente (costo $c = 1$) modifica la distancia en línea recta hacia cualquier objetivo en como máximo 1 unidad (de hecho, suele ser < 1). En los cambios de fase se cumple la condición trivialmente al llegar a la celda objetivo ($h = 0$).

Código corregido y optimizado

```
def _euclideanDistance(a, b):
    """Distancia euclidiana sqrt(dx^2 + dy^2) entre dos celdas."""
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def _phaseDistance(state, problem, distance):
    """
    Aplica la regla de fase y devuelve la distancia al objetivo obligatorio.

    state      : (position, hasKit, pendingSystems)
    problem     : SystemRepairProblem (aporta kitPosition y controlPosition)
    distance    : funcion de distancia a usar (_manhattanDistance o _euclideanDistance)
    """

    position, hasKit, pendingSystems = state

    if not hasKit:
        return distance(position, problem.kitPosition)

    if pendingSystems:
        return min(distance(position, system) for system in pendingSystems)

    return distance(position, problem.controlPosition)
```

```
def euclideanHeuristic(state, problem):
    """
    Punto 5c. Igual que la anterior pero con distancia euclidiana.

    Tambien es admisible y consistente, por el mismo argumento con la norma
    euclidiana (un paso desplaza exactamente 1 en esa norma).

    Es ESTRICTAMENTE MAS DEBIL que Manhattan: como no hay movimiento diagonal,
    siempre se cumple euclidiana <= Manhattan <= costo real. Por eso se
    espera que expanda mas nodos que manhattanHeuristic en todos los mapas.
    """
    return _phaseDistance(state, problem, _euclideanDistance)
```

¿Por qué es mejor esta versión?

Esta versión es superior porque aplica el patrón de diseño *Strategy* al desacoplar la lógica de transición entre las fases del problema del cálculo geométrico de la distancia, centralizando las reglas de negocio en la función `_phaseDistance` y garantizando el cumplimiento del principio *DRY* (Don't Repeat Yourself). Al pasar la métrica de distancia como una función de orden superior, la arquitectura del sistema gana alta reusabilidad y mantenibilidad, permitiendo incorporar nuevas métricas de navegación o modificar el número de fases sin duplicar código ni alterar la estructura principal. Además, aporta un valioso rigor técnico al documentar la relación de dominancia $h_{euclidiana}(n) \leq h_{manhattan}(n) \leq h^*(n)$, explicando formalmente que, al no contemplar desplazamientos diagonales en una cuadrícula ortogonal, la heurística euclidiana es estrictamente más débil que la de Manhattan y derivará en una mayor expansión de nodos durante la búsqueda con algoritmos como A^* .

d)

Código Inicial SystemRepair:

```
def systemRepairHeuristic(
    state: Tuple[Tuple, bool, Tuple], problem: SystemRepairProblem
):
    """
    """
    position, hasKit, pendingSystems = state

    if problem.isGoalState(state):
        return 0

    def compute_mst(nodes):
        if len(nodes) <= 1:
            return 0

        key = tuple(sorted(nodes))
        if hasattr(problem, 'heuristicInfo') and key in problem.heuristicInfo:
            return problem.heuristicInfo[key]

        n = len(nodes)
        visited = [False] * n
        min_dist = [float('inf')] * n
        min_dist[0] = 0
        total_mst = 0

        for _ in range(n):
            u = -1
            best_d = float('inf')
            for i in range(n):
                if not visited[i] and min_dist[i] < best_d:
                    best_d = min_dist[i]
                    u = i

            if u == -1:
                break
```

```

visited[u] = True
total_mst += best_d

pu = nodes[u]
for v in range(n):
    if not visited[v]:
        pv = nodes[v]
        d = abs(pu[0] - pv[0]) + abs(pu[1] - pv[1])
        if d < min_dist[v]:
            min_dist[v] = d

if hasattr(problem, 'heuristicInfo'):
    problem.heuristicInfo[key] = total_mst

return total_mst

if not hasKit:
    dist_to_kit = abs(position[0] - problem.kitPosition[0]) + abs(
        position[1] - problem.kitPosition[1]
    )
    nodes = (problem.kitPosition,) + pendingSystems + (problem.controlPosition,)
    return dist_to_kit + compute_mst(nodes)
else:
    nodes = (position,) + pendingSystems + (problem.controlPosition,)
    return compute_mst(nodes)

```

Prompt:

(Código system repair adjunto)

Toca implementar la heurística de la distancia

Necesito que:

1. Dime si el código es correcto (admisible y consistente).
2. Si no es correcto, dime exactamente qué falla y dame la corrección.
3. Si es correcto, pero existe una versión mejor, dame la mejor y la razón de por qué es mejor.

Respuesta:

Propiedades de la heurística

- Admisible. Sí. $\text{Manhattan} \leq \text{distancia real}$, así que el MST con pesos Manhattan es una cota inferior del MST real, que a su vez es una cota inferior del recorrido que falta. Todo lo que subestima sigue siendo válido. Por eso A* devuelve el óptimo con ella en todos los mapas donde alcanza a terminar.
- Consistente. No. La causa está en la rama else: metes position como nodo del árbol. Cuando el robot da un paso, la posición cambia y el MST se reestructura: al sacar el nodo viejo y meter el nuevo, las aristas del árbol se recalculan completas y el peso total puede caer más de 1 en un solo movimiento. Eso rompe la desigualdad. Conviene precisar que el origen del problema no es la métrica empleada sino un defecto estructural.

Código corregido y optimizado

```
def _mapDistances(problem, source):
    """
    Distancias reales (minimo numero de pasos, esquivando muros) desde `source`
    hacia todas las celdas transitables del mapa. Es un BFS estandar.

    El resultado se memoiza en problem.heuristicInfo["mapDistances"], porque la
    topologia del mapa no cambia durante la busqueda: basta un BFS por punto de
    interes (K, C y cada T), es decir a lo sumo  $|T| + 2$  recorridos  $O(\text{celdas})$ .
    """

    cache = problem.heuristicInfo.setdefault("mapDistances", {})
    if source in cache:
        return cache[source]

    walls = problem.walls
    distances = {source: 0}
    queue = deque([source])

    while queue:
        current = queue.popleft()
        currentDistance = distances[current]

        for direction in [
            Directions.NORTH,
            Directions.SOUTH,
            Directions.EAST,
            Directions.WEST,
        ]:
            dx, dy = Actions.directionToVector(direction)
            neighbour = (int(current[0] + dx), int(current[1] + dy))

            if walls[neighbour[0]][neighbour[1]]:
                continue
            if neighbour in distances:
                continue
```

```
            distances[neighbour] = currentDistance + 1
            queue.append(neighbour)

    cache[source] = distances
    return distances

def _mapDistance(problem, poi, cell):
    """
    Distancia real entre un punto de interes `poi` (K, C o un T) y una celda
    cualquiera `cell`. UNREACHABLE si no hay camino.

    Importante: el BFS SIEMPRE se lanza desde el punto de interes, nunca desde
    la celda donde esta el robot. Como la distancia es simetrica el resultado
    es el mismo, pero asi el numero de BFS distintos queda acotado por
     $|T| + 2$  en vez de crecer con el numero de celdas visitadas.
    """

    return _mapDistances(problem, poi).get(cell, UNREACHABLE)
```

```

def _minimumSpanningTree(problem, nodes):

    cache = problem.heuristicInfo.setdefault("mstCache", {})
    if nodes in cache:
        return cache[nodes]

    nodeList = list(nodes)
    size = len(nodeList)

    if size <= 1:
        cache[nodes] = 0
        return 0

    distance = [[0] * size for _ in range(size)]
    for i in range(size):
        fromNode = _mapDistances(problem, nodeList[i])
        for j in range(size):
            distance[i][j] = fromNode.get(nodeList[j], UNREACHABLE)

    inTree = [False] * size
    cheapest = [UNREACHABLE] * size
    cheapest[0] = 0
    total = 0

    for _ in range(size):
        chosen = -1
        for candidate in range(size):
            if not inTree[candidate]:
                if chosen == -1 or cheapest[candidate] < cheapest[chosen]:
                    chosen = candidate

        inTree[chosen] = True
        total += cheapest[chosen]

```

```

def _minimumSpanningTree(problem, nodes):
    for other in range(size):
        if not inTree[other] and distance[chosen][other] < cheapest[other]:
            cheapest[other] = distance[chosen][other]

    cache[nodes] = total
    return total

```

```
def systemRepairHeuristic(
    state: Tuple[Tuple, bool, Tuple], problem: SystemRepairProblem
):
    """
    Punto 5d. Heurística propia para SystemRepairProblem.

    Formula
    -----
    
$$h(n) = \text{prefijo} + \max(\text{entrada} + \text{MST}(R), \max_t [d(a,t) + d(t,C)] )$$


    donde
    R      = {T pendientes} U {C}
    prefijo = d(p, K) si aun no tiene el kit, 0 si ya lo tiene
    a      = K si aun no tiene el kit, la posicion actual si ya lo tiene
    entrada = min_{x in R} d(a, x)
    d      = distancia REAL sobre el mapa (BFS, esquiva muros)

    """

    position, hasKit, pendingSystems = state
    controlPosition = problem.controlPosition

    if hasKit and not pendingSystems and position == controlPosition:
        return 0

    if hasKit:
        prefix = 0
        anchor = position
    else:
        prefix = _mapDistance(problem, problem.kitPosition, position)
        anchor = problem.kitPosition

    required = pendingSystems + (controlPosition,)

    anchorDistance = {node: _mapDistance(problem, node, anchor) for node in required}
```

```
def systemRepairHeuristic(
    entryCost = min(anchorDistance.values())
    bound = entryCost + _minimumSpanningTree(problem, required)

    for system in pendingSystems:
        candidate = anchorDistance[system] + _mapDistance(
            problem, controlPosition, system
        )
        if candidate > bound:
            bound = candidate

    return prefix + bound
```

¿Por qué es mejor esta versión?

La corrección consiste en excluir la posición del robot del árbol y contabilizarla por separado. Sea $R = \{\text{sistemas pendientes}\} \cup \{C\}$ y sea a la posición actual del robot cuando ya dispone del kit, o la celda K en caso contrario. La heurística corregida se expresa como $h(n) = \text{prefijo} + \min_{x \in R} d(a, x) + \text{MST}(R)$, donde el prefijo corresponde a la distancia hasta K cuando el kit aún no ha sido recogido y es nulo en caso contrario. Esta reformulación resuelve simultáneamente tres deficiencias. En primer lugar, elimina la inconsistencia. En segundo lugar, **fortalece** la cota, puesto que se cumple $\text{MST}(\{a\} \cup R) \leq \min_{x \in R} d(a, x) +$

MST(R), es decir, separar el término de entrada nunca produce una estimación menor. En tercer lugar, corrige un problema de rendimiento no evidente: al depender el árbol únicamente del conjunto de sistemas pendientes, su valor puede memorizarse con dicho conjunto como llave y solo requiere recalcularse cuando se repara un sistema.

Una vez restablecida la consistencia, la diferencia restante con la versión definitiva es exclusivamente de fuerza estimativa y proviene de la métrica utilizada como peso de las aristas. En mapas de topología laberíntica, dos sistemas separados por un muro extenso presentan una distancia Manhattan de unas pocas unidades frente a una distancia real de varias decenas de pasos, de modo que el árbol subestima gravemente el recorrido pendiente justo en las instancias donde una estimación ajustada resulta más necesaria. La versión definitiva sustituye la distancia Manhattan por distancias reales precalculadas mediante BFS desde cada punto de interés —K, C y cada sistema averiado, es decir $|T| + 2$ recorridos memorizados— y añade una segunda cota inferior de costo lineal, dada por $\max_{t \in \text{pendientes}} [d(a,t) + d(t,C)]$, que domina cuando existe un sistema notablemente alejado del resto.

Punto 6

a)

Complejidades

algoritmo	Complejidad temporal	Complejidad espacial
DFS	$O(V+E)$	$O(V)$
BFS	$O(V+E)$	$O(V)$
UCS	$O(E \log E)$	$O(V+E)$
A*	$O(b^d)$	$O(b^d)$

Complejidad

algoritmo	¿Es completo?	Condiciones para encontrar una solución óptima
DFS	Si	Si sólo existe una sola ruta posible desde R hasta D y si la meta es el primer nodo que visita.
BFS	Si	Necesita que todos los pasos tengan el mismo peso.
UCS	Si	Que los costos no tengan números negativos. Sin embargo, en la estación espacial no los hay.
A*	Si	Depende de la Heurística.

Optimalidad

Algoritmo	¿Es óptimo?	Condición
DFS	No	Ninguna
BFS	Si	Todas las acciones cuentan igual (falla cuando el costo de paso no es uniforme).
UCS	Si	Costos no negativos
A*	Si	Su heurística es admisible (en búsqueda en árboles) o consistente (en búsqueda en grafos).

Consistencia de las Heurísticas

manhattanHeuristic es consistente. Dentro de una misma fase de la misión, h es la distancia Manhattan, o el mínimo de varias distancias Manhattan, a un conjunto fijo de celdas; como cada movimiento desplaza al robot exactamente una unidad, esa distancia varía a lo sumo en 1, de donde $h(n) \leq 1 + h(n')$. La fase solo cambia cuando el robot entra en K o en un sistema pendiente, y en ese instante el término que desaparece vale exactamente 1, porque el estado sucesor es precisamente esa celda. Es también admisible, porque el objetivo de cada fase es de visita obligatoria y la distancia Manhattan nunca supera la distancia real en una cuadrícula sin movimientos diagonales.

euclideanHeuristic es consistente porque cumple con la desigualdad triangular: la distancia en línea recta entre cualquier nodo A y el objetivo C siempre es menor o igual que la suma del costo de ir de A a un vecino intermedio B más la distancia estimada desde B hasta el objetivo. Sin embargo, es estrictamente más débil que Manhattan: dado que no existe movimiento diagonal, siempre se cumple la cadena euclidiana $\leq$ Manhattan $\leq$ costo real, de modo que Manhattan domina a euclidiana.

systemRepairHeuristic es admisible y consistente. Su formulación es

$$h(n) = \text{prefijo} + \max(\text{entrada} + \text{MST}(R), \max_{t \in \text{pendientes}} [d(a,t) + d(t,C)])$$

donde $R = \{\text{sistemas pendientes}\} \cup \{C\}$, el *prefijo* es $d(p,K)$ cuando el kit aún no se ha recogido y 0 en caso contrario, a es la posición actual o la celda K según corresponda, $\text{entrada} = \min_{x \in R} d(a,x)$, y todas las distancias d son distancias reales sobre el mapa calculadas por BFS.

Es admisible porque los tres tramos que suma son obligatorios y disjuntos en el tiempo: sin el kit ninguna reparación se registra, luego el tramo hasta K es forzoso; desde a el recorrido debe alcanzar algún nodo de R antes de poder avanzar en la misión, lo que cuesta al menos el mínimo de esas distancias; y el resto del recorrido visita todos los nodos de R , de modo que sus aristas forman un subgrafo conexo sobre R y pesan al menos $\text{MST}(R)$. La suma de cotas inferiores de

tramos disjuntos sigue siendo cota inferior, y el máximo de dos cotas inferiores también lo es. En un estado meta se tiene $R = \{C\}$, con $MST(\{C\}) = 0$ y $\min d(C, x) = 0$, luego $h = 0$, como exige el enunciado.

Es consistente porque las distancias reales satisfacen la desigualdad triangular y cada paso cuesta 1, de manera que cada término de distancia varía a lo sumo en una unidad por movimiento. El término $MST(R)$ **no depende de la posición del robot**, por lo que permanece constante durante un movimiento ordinario; y cuando sí cambia, porque se reparó un sistema, el término que desaparece vale exactamente 1, obteniéndose de nuevo la igualdad $h(n) = 1 + h(n')$.

Costo de la solución

DFS, BFS y UCS:

Mapas	T	DFS	BFS	UCS
tinyRepair	1	92	24	24
brokenRing	4	252	94	94
maintenanceWeb	6	539	117	117
corridorCluster	9	1243	181	181
finalMaintenanceComplex	15	2729	-	-

BFS y UCS devuelven exactamente el mismo costo en los cuatro mapas donde ambos terminan, mientras que DFS devuelve caminos entre 3.8 y 9.6 veces más costosos. La brecha crece con el tamaño del mapa. Esto no es una casualidad de estos mapas sino la consecuencia directa de que DFS devuelve el primer camino que encuentra, y cuanto más grande es el espacio de búsqueda, más lejos queda ese primer camino del óptimo.

DFS logra terminar el mapa de finalMaintenanceComplex porque su trabajo es proporcional a la longitud del camino que encuentra, mientras que el de BFS y UCS no lo terminan porque son proporcional al volumen completo del espacio de estados hasta la profundidad de la solución óptima, es decir, recorren un espacio que crece exponencialmente con el número de sistemas averiados, ya que el estado incluye el subconjunto de sistemas aún pendientes. A esto se suma un factor de memoria decisivo: la frontera de DFS es una pila cuyo tamaño crece apenas con la profundidad, mientras que la de BFS y UCS contiene un nivel completo del árbol de búsqueda, con cada entrada arrastrando además la lista de acciones que conduce hasta ella, de manera que el proceso agota la memoria disponible mucho antes de acercarse a la profundidad de la solución.

A*:

En los treinta mapas de layouts/systems, todas las heurísticas que logran terminar devuelven exactamente el mismo costo. Esto es lo esperado y constituye la primera verificación de correctitud: las cuatro heurísticas son admisibles, de modo que A* garantiza la solución óptima con cualquiera de ellas. Una heurística no mejora la calidad de la respuesta; únicamente reduce el trabajo necesario para encontrarla. Si alguna hubiera devuelto un costo distinto, eso habría revelado una violación de admisibilidad.

Nodos Expandidos:

DFS, BFS y UCS:

Mapas	T	DFS	BFS	UCS
tinyRepair	1	92	24	24
brokenRing	4	252	94	94
maintenanceWeb	6	539	117	117
corridorCluster	9	1243	181	181
finalMaintenanceComplex	15	2729	-	-

A*

Mapas	T	Costo	Null	Manhattan	Euclidean	Propia
tinyRepair	1	24	148	127	129	35
brokenRing	4	94	2950	2636	2671	871
maintenanceWeb	6	117	16147	13620	14076	387
corridorCluster	9	181	190045	187992	188663	20992
finalMaintenanceComplex	15					248433

Las primeras tres heurísticas solo se ejecutaron hasta $|T| = 9$; a partir de diez sistemas averiados agotan la memoria antes de terminar. La causa es estructural: el espacio de estados crece exponencialmente en el número de sistemas averiados, porque el estado incluye el subconjunto de sistemas pendientes. Con una heurística que estima solo una fracción pequeña del costo restante, A* degenera en UCS y debe expandir una porción enorme de ese espacio; a ello se suma que cada entrada de la frontera almacena la lista de acciones que conduce hasta ella, lo que multiplica el consumo de memoria por la longitud del camino. La heurística propia evita el colapso porque su cota crece con el número de sistemas pendientes: cuando quedan muchos objetivos dispersos, su valor aumenta y A* descarta de inmediato las ramas que postergan los sistemas más alejados. Esa es exactamente la información que las heurísticas base no capturan.

NullHeuristic no aporta información alguna, de modo que A* se comporta exactamente como UCS y expande todos los estados cuyo costo acumulado es menor que el óptimo. manhattanHeuristic y euclideanHeuristic reducen las expansiones de forma marginal, porque solo estiman la distancia hasta el siguiente objetivo obligatorio e ignoran por completo cuántos sistemas quedan pendientes. La heurística propia reduce las expansiones en varios órdenes de magnitud, porque estima el recorrido completo que falta en lugar de una sola etapa.

Tiempo de ejecución (segundos)

DFS, BFS y UCS:

Mapas	T	DFS	BFS	UCS
tinyRepair	1	0,0	0,0	0,0
brokenRing	4	0,0	0,1	0,1

maintenanceWeb	6	0,0	0,2	0,3
corridorCluster	9	0,0	4,7	3,1
finalMaintenanceComplex	15	0,3	-	-

A*:

Mapas	Null	Manhattan	Euclidean	Propia
tinyRepair	0,0	0,0	0,0	0,0
brokenRing	0,0	0,0	0,0	0,0
maintenanceWeb	0,1	0,1	0,1	0,0
corridorCluster	1,3	2,4	1,3	0,2
finalMaintenanceComplex				5,1

ManhattanHeuristic y euclideanHeuristic tardan más que nullHeuristic pese a expandir menos nodos. La explicación está en que toda heurística tiene un costo de evaluación que se paga en cada sucesor generado, y cuando el ahorro en exploración es pequeño ese costo no se recupera. La heurística propia es la única cuyo tiempo total disminuye respecto a la línea base, porque su ganancia en exploración supera ampliamente lo que cuesta calcularla, y ese cómputo, además, está amortizado mediante la memorización del árbol de recubrimiento y de las distancias reales del mapa.

Conclusión:

Los cuatro algoritmos implementados no constituyen una escala de mejora progresiva, sino un conjunto de compromisos distintos entre completitud, optimalidad y consumo de recursos, y la elección correcta depende del criterio de optimalidad que exija la misión. DFS es completo pero no óptimo bajo ningún criterio: expande pocos nodos y termina incluso en las instancias más grandes, aunque solo porque se detiene ante el primer camino hallado, devolviendo soluciones cuyo costo excede varias veces el óptimo. BFS añade la primera garantía — optimalidad en número de movimientos— pero la pierde en costo en cuanto el terreno deja de ser homogéneo, pues prefiere atajos que atraviesan las celdas más caras. UCS traslada el criterio de optimalidad al costo total y resuelve así el problema que BFS no puede abordar, a cambio de un sobre costo modesto; su limitación es estructural, ya que al carecer de información sobre el costo restante debe descartar todos los candidatos de costo inferior antes de certificar el óptimo, trabajo que crece exponencialmente con el número de objetivos pendientes hasta volverse inabordable en memoria.

A* supera esa limitación conservando la optimalidad de UCS y reduciendo la exploración necesaria para certificarla, pues no modifica el criterio de optimalidad sino el orden de exploración: mientras la heurística sea admisible, la garantía permanece intacta y lo único que cambia es cuántas ramas quedan descartadas sin expandirse. De ahí la conclusión central del taller: la admisibilidad determina la corrección del resultado, mientras que el grado de información de la heurística determina el trabajo necesario para obtenerlo. Las heurísticas de distancia directa, siendo admisibles y consistentes, apenas reducen las expansiones porque estiman una sola etapa de una misión cuya dificultad reside en el orden de visita de múltiples

objetivos; la heurística propia, igualmente admisible y consistente pero construida sobre una relajación del recorrido completo a un árbol de recubrimiento mínimo con distancias reales del mapa, convierte instancias intratables en problemas de segundos sin sacrificar la optimalidad. En términos del escenario, el robot de la estación Aurora no requiere el algoritmo que explore menos ni el que responda más rápido, sino aquel que garantice la ruta de costo mínimo dentro del tiempo y la memoria disponibles, y solo A* con una heurística bien informada satisface ambas exigencias.

b) Reflexión sobre el uso de inteligencia artificial:

Las correcciones que propuso la IA fueron desde muy básicas hasta bastante complejas. Cada vez soluciona errores de manera más contundente. Basta con poner en el prompt el tipo de error que sale en la terminal y el código en cuestión para que encuentre la solución. Esto es un cambio, pues en versiones anteriores de IA no era capaz de solucionarlo y entraba en un bucle de respuestas que no llevaban a nada, e incluso repetía soluciones que no funcionaban. A parte, logra identificar de manera correcta cuando una heurística es consistente o admisible. Sin embargo, lo más útil fue las sugerencias de la IA que proponían mejoras en el algoritmo, puesto que siempre recomendaba métodos poco convencionales o que no siempre se ven en clase, producto del proceso de aprendizaje de la IA todos estos años. Resulta ser eficiente a la hora de optimizar algoritmos, pues detecta patrones lógicos claros que mejora con otro tipo de técnicas que son de gran ayuda. Además, son soluciones que no serían tan difíciles de explicar, puesto que se tratan de estrategias que no son difíciles de implementar, pero si difíciles de encontrar a la primera o pensarlas. En cuanto al prompt no fue difícil decidir el texto correcto. Parece que cada vez la IA necesita contextos menos amplios para dar soluciones ajustadas a lo que se requiere.